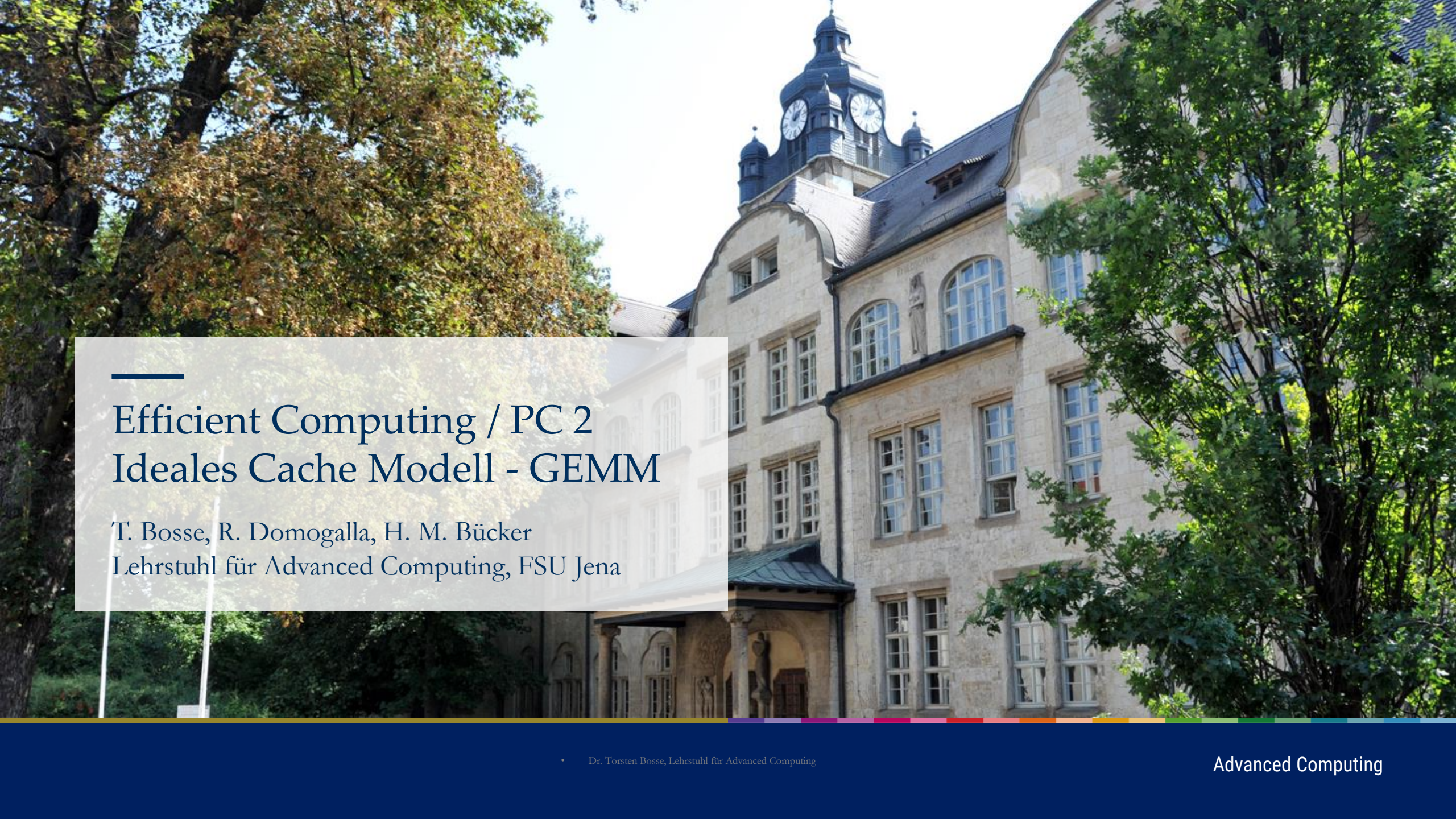




# Efficient Computing / PC 2

## Ideales Cache Modell - GEMM

T. Bosse, R. Domogalla, H. M. Bucker  
Lehrstuhl für Advanced Computing, FSU Jena



# Motivation

- **Development Memory vs. CPU**
- **Neumann-Bootleneck**
- **Cache (Hierarchy, Strategy,...)**
- **Ideal Cache Model**
- **Cache-Aware Algorithms**

# MULT: Scalar Version

---

**Algorithm 1.1:** Given three  $n \times n$  matrices  $Z$ ,  $X$  and  $Y$ , compute the update  $Z \leftarrow Z + X \cdot Y$  using an ordinary scalar algorithm.

---

```
1 Function  $Z = \text{MULT}(X, Y, Z, n)$ 
2   for  $i = 1$  to  $n$  do
3     for  $j = 1$  to  $n$  do
4       for  $k = 1$  to  $n$  do
5          $Z(i, j) \leftarrow Z(i, j) + X(i, k) \cdot Y(k, j)$ 
```

---

# Linear Storage: Column-Major Order

|    |    |    |    |    |    |    |    |    |  |  |  |
|----|----|----|----|----|----|----|----|----|--|--|--|
| 1  | 13 | 25 | 37 | 49 | 61 | 73 | 85 | 97 |  |  |  |
| 2  | 14 | 26 | 38 | 50 | 62 | 74 | 86 | 98 |  |  |  |
| 3  | 15 | 27 | 39 | 51 | 63 | 75 | 87 | 99 |  |  |  |
| 4  | 16 | 28 | 40 | 52 | 64 | 76 | 88 |    |  |  |  |
| 5  | 17 | 29 | 41 | 53 | 65 | 77 | 89 |    |  |  |  |
| 6  | 18 | 30 | 42 | 54 | 66 | 78 | 90 |    |  |  |  |
| 7  | 19 | 31 | 43 | 55 | 67 | 79 | 91 |    |  |  |  |
| 8  | 20 | 32 | 44 | 56 | 68 | 80 | 92 |    |  |  |  |
| 9  | 21 | 33 | 45 | 57 | 69 | 81 | 93 |    |  |  |  |
| 10 | 22 | 34 | 46 | 58 | 70 | 82 | 94 |    |  |  |  |
| 11 | 23 | 35 | 47 | 59 | 71 | 83 | 95 |    |  |  |  |
| 12 | 24 | 36 | 48 | 60 | 72 | 84 | 96 |    |  |  |  |

# Linear Storage: Row-Major Order

[illegible]

# MULT: Scalar Version

---

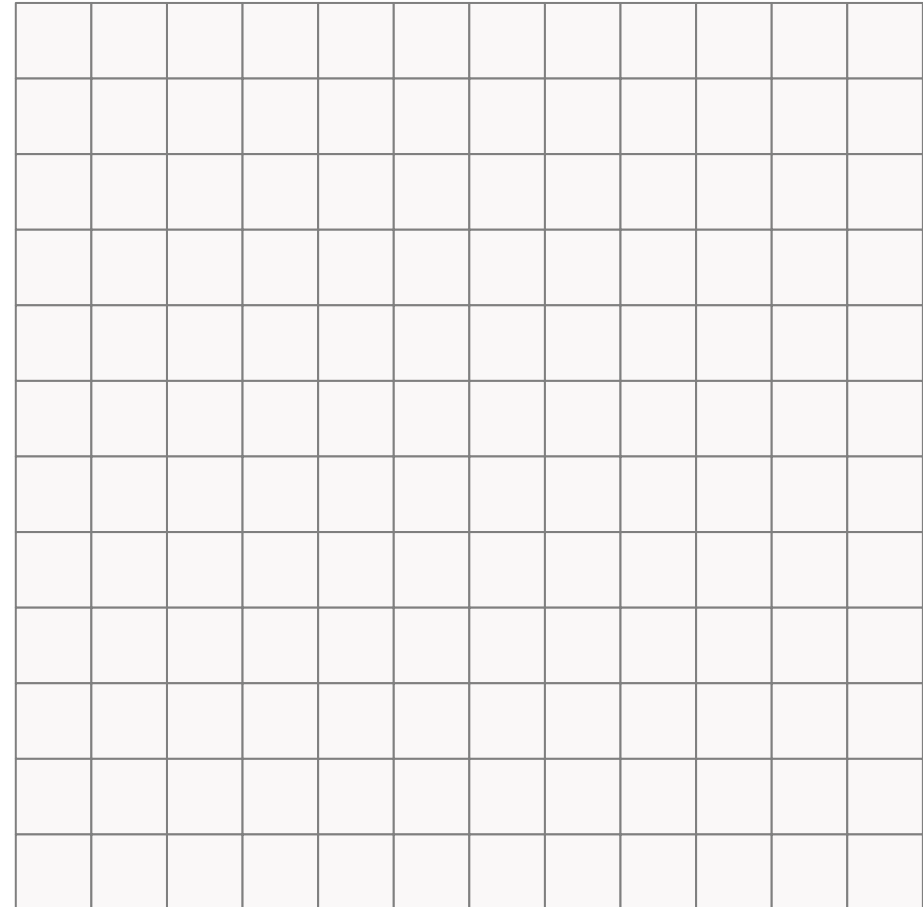
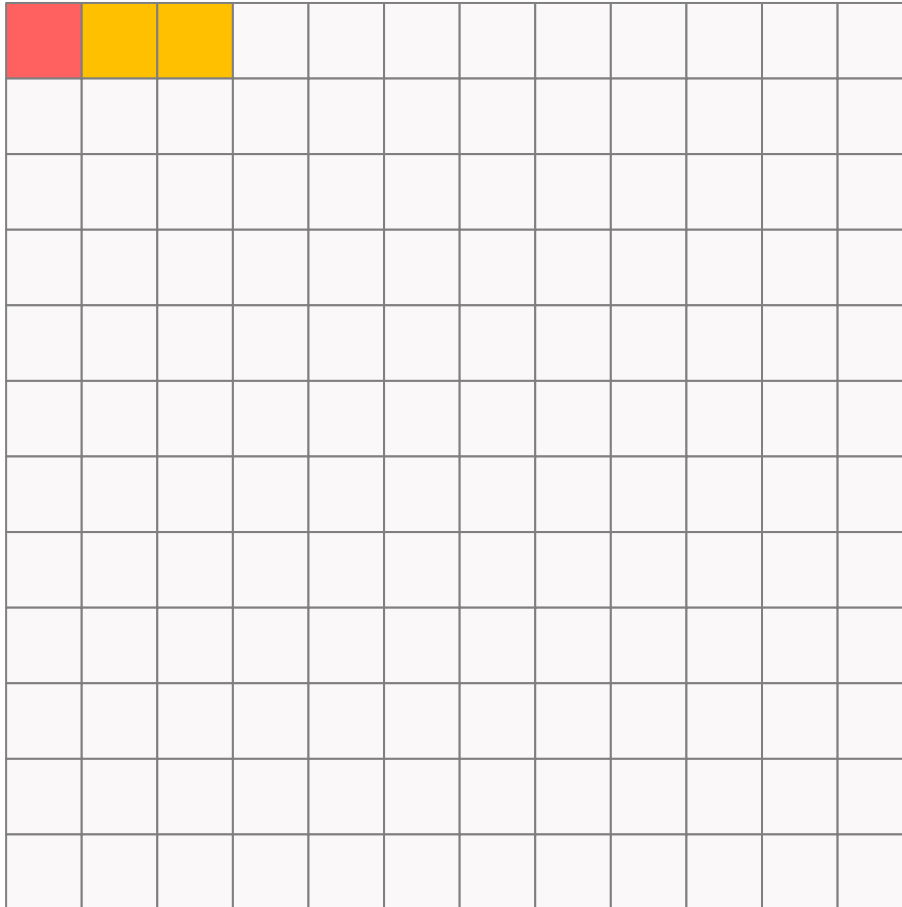
**Algorithm 1.1:** Given three  $n \times n$  matrices  $Z$ ,  $X$  and  $Y$ , compute the update  $Z \leftarrow Z + X \cdot Y$  using an ordinary scalar algorithm.

---

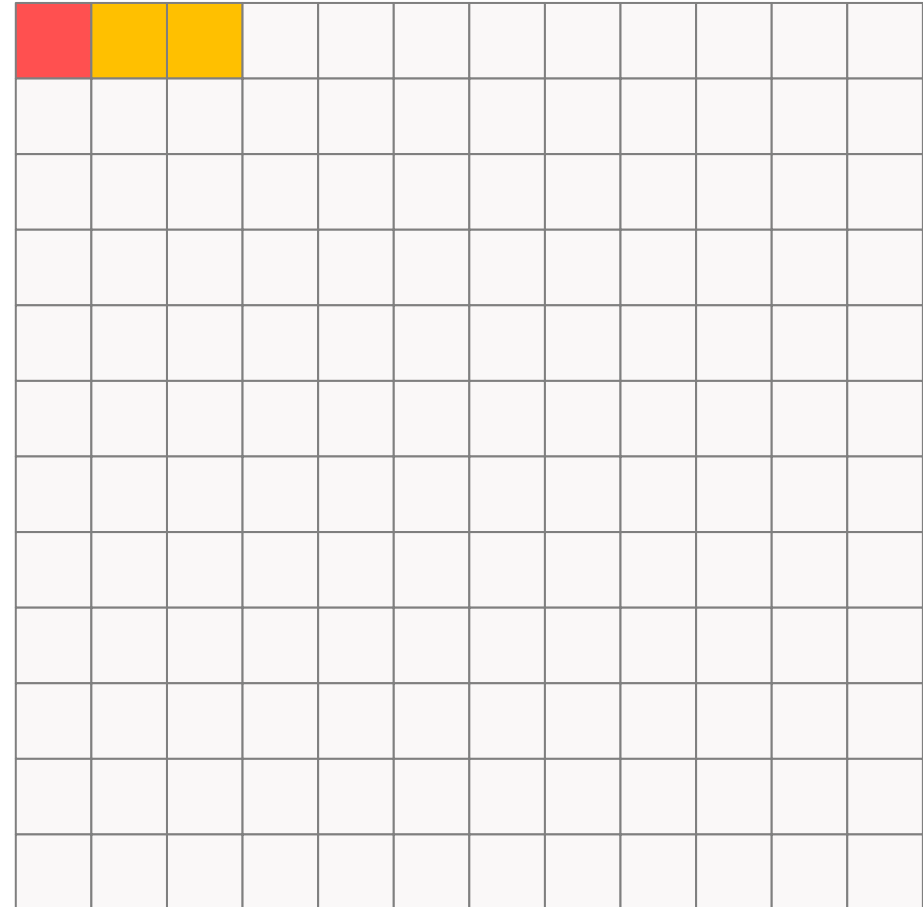
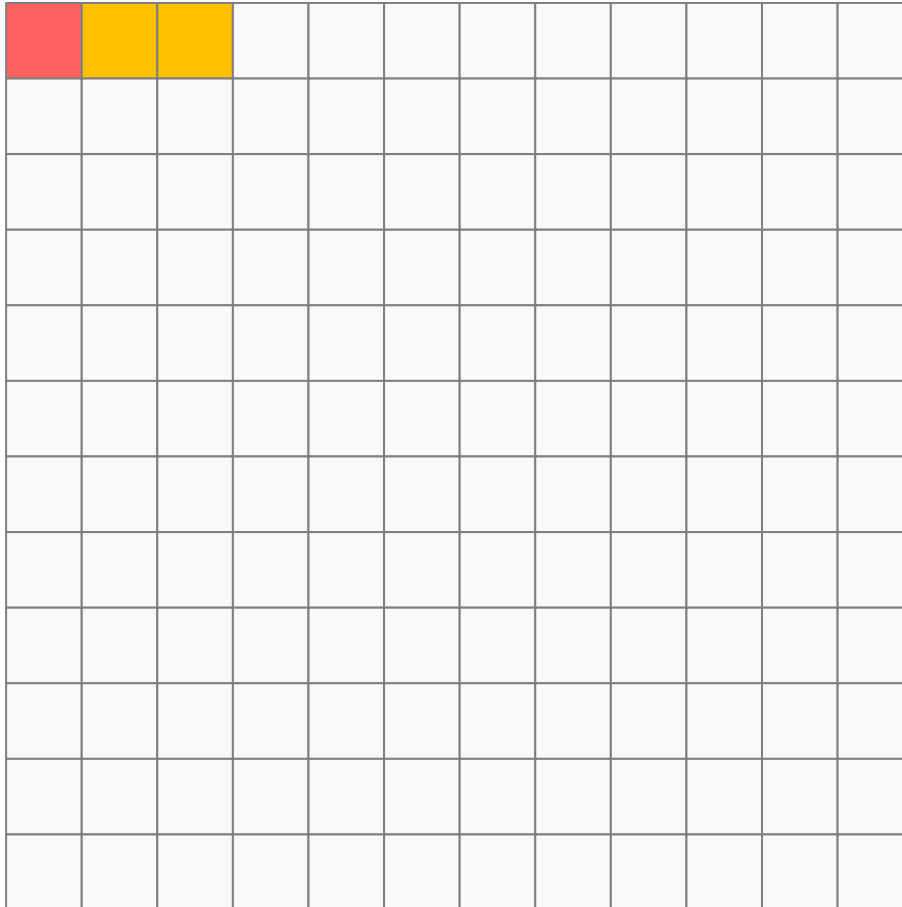
```
1 Function  $Z = \text{MULT}(X, Y, Z, n)$ 
2   for  $i = 1$  to  $n$  do
3     for  $j = 1$  to  $n$  do
4       for  $k = 1$  to  $n$  do
5          $Z(i, j) \leftarrow Z(i, j) + X(i, k) \cdot Y(k, j)$ 
```

---

# MULT: Memory Accesses

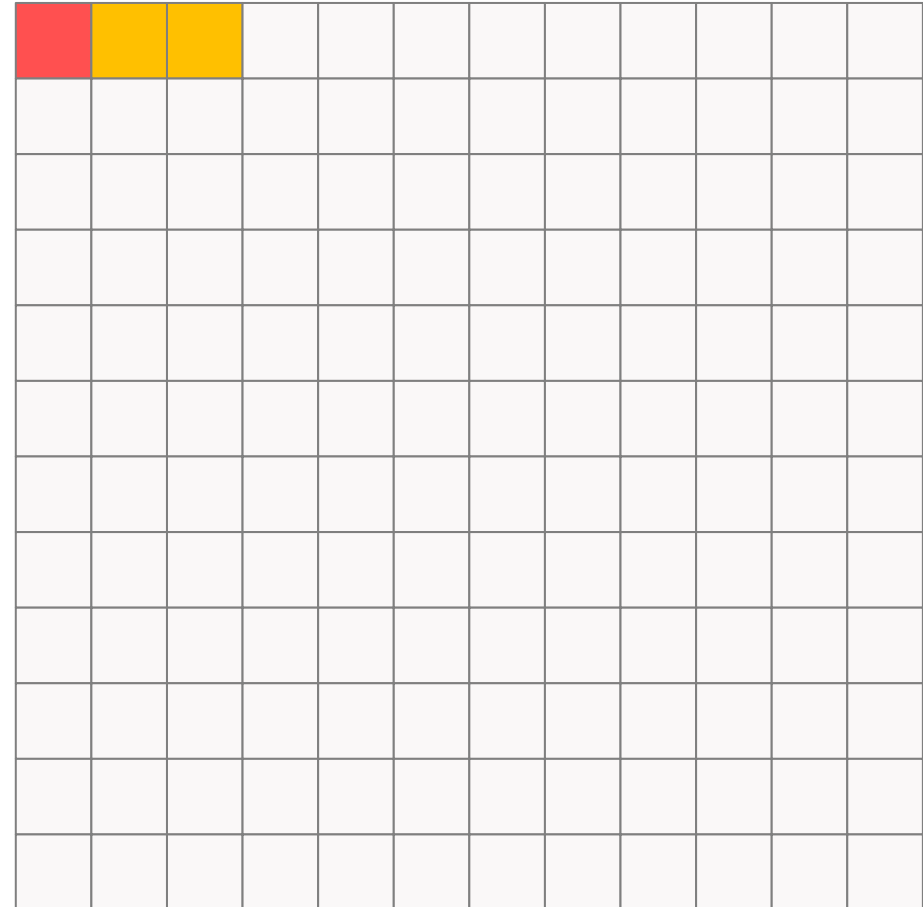
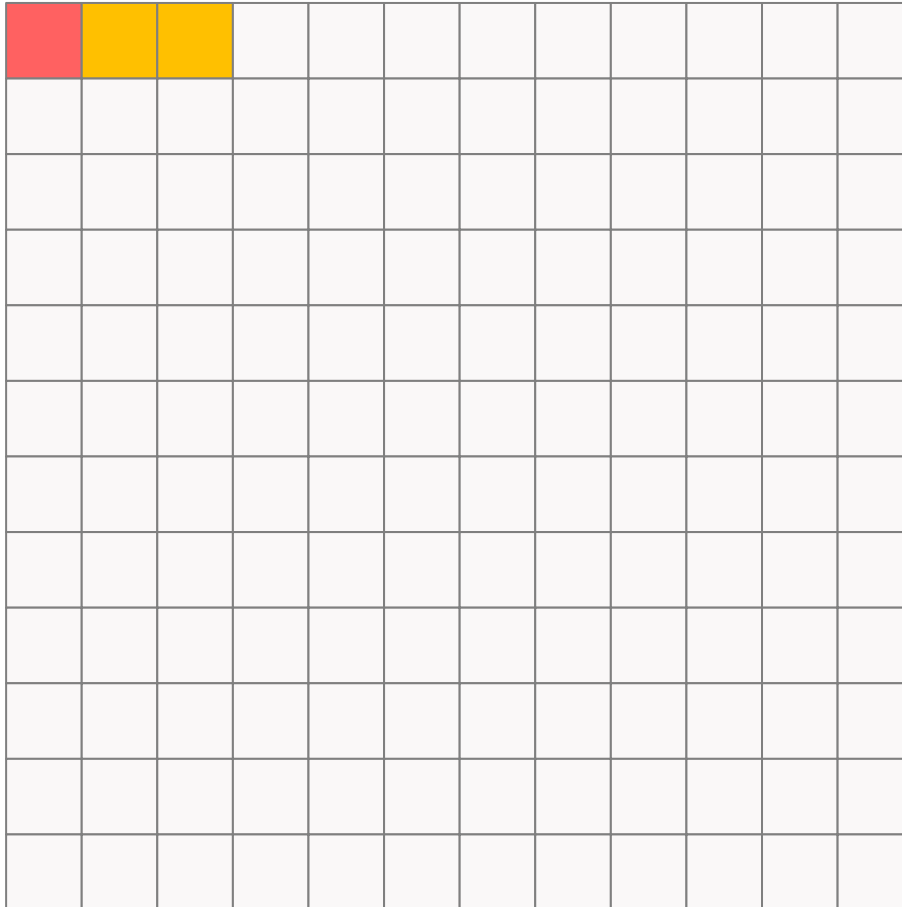
$$X(i, 1) \ Y(1, j) \ X(i, 2) \ Y(2, j) \ \dots \ X(i, B) \ Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


# MULT: Memory Accesses

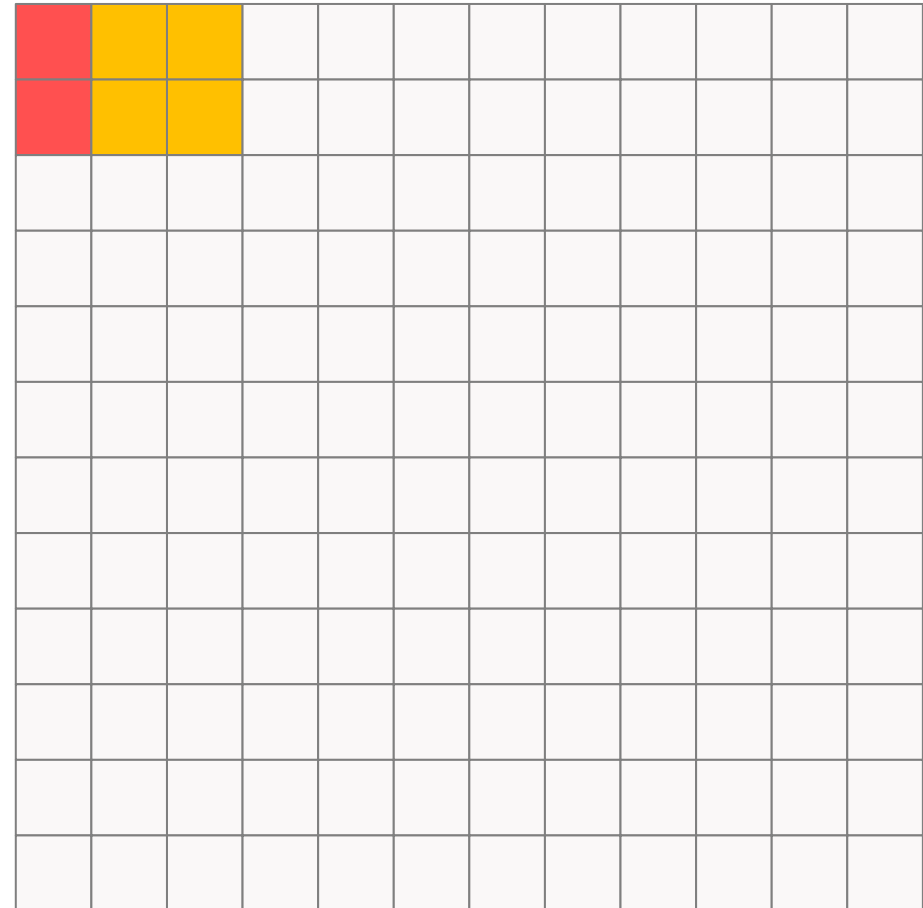
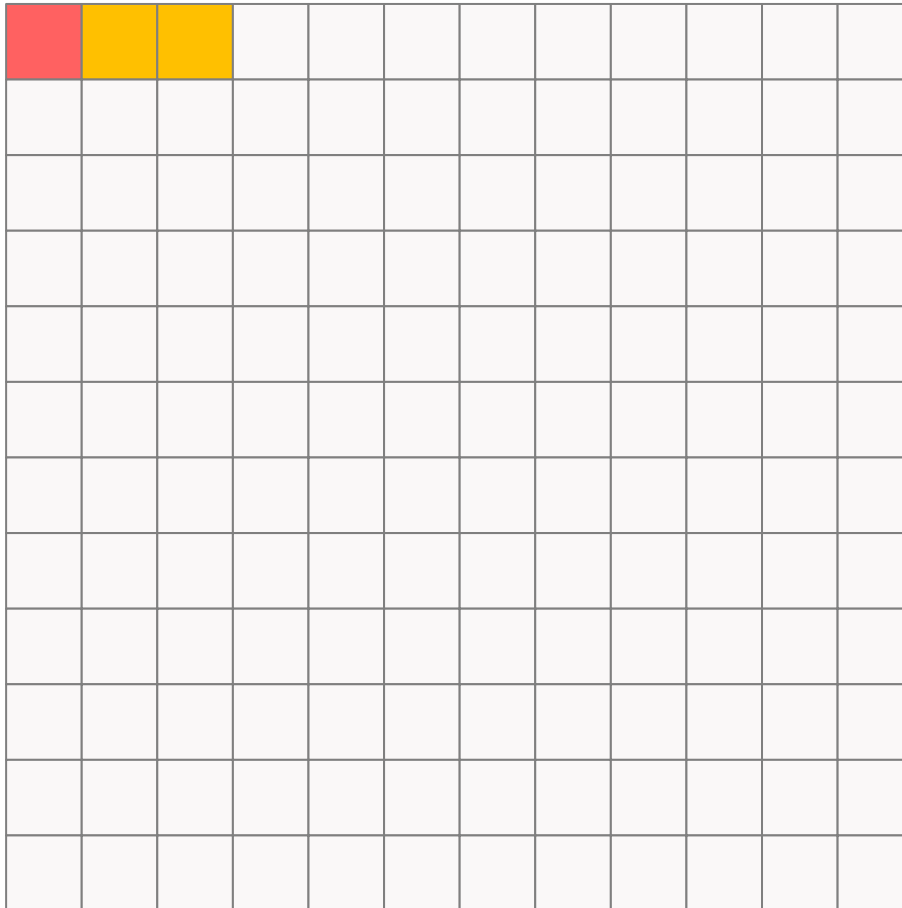
$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$




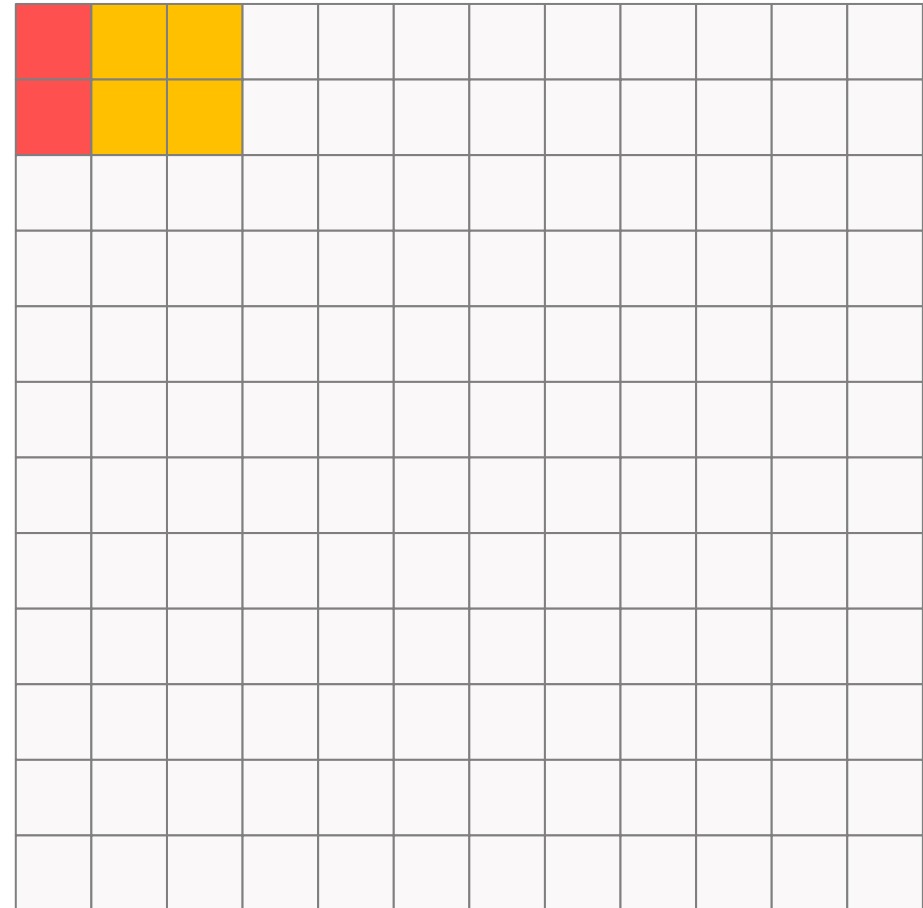
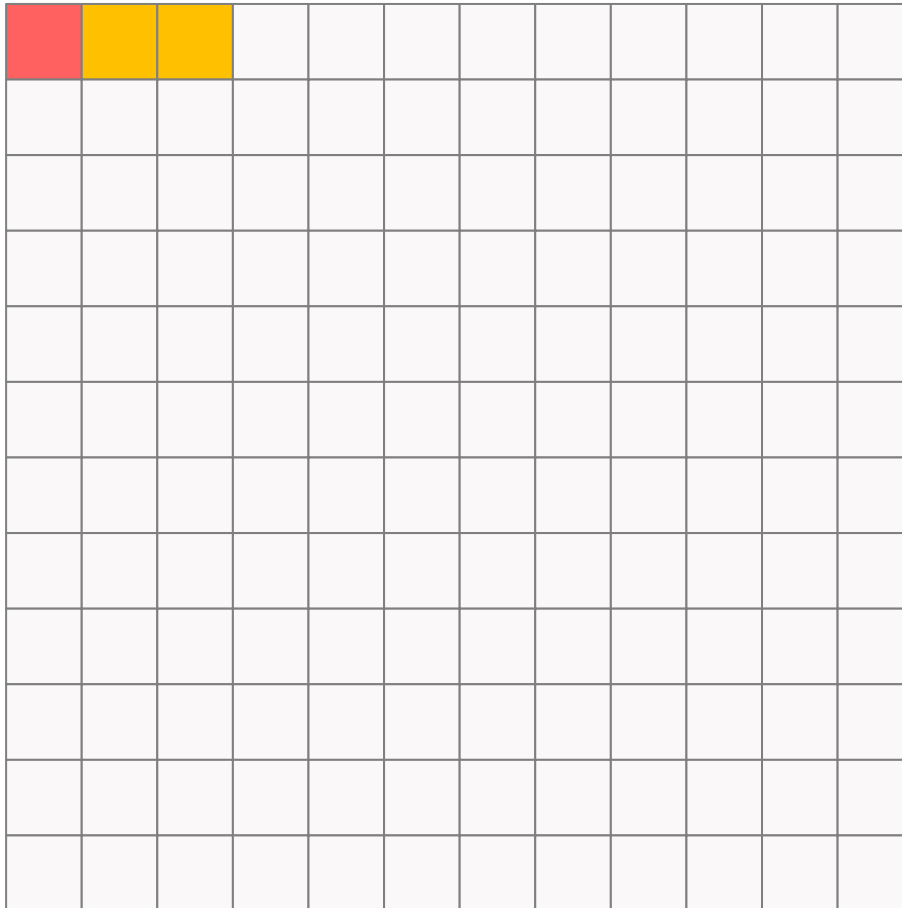
# MULT: Memory Accesses

$$X(i, 1) \, Y(1, j) \, X(i, 2) \, Y(2, j) \, \dots \, X(i, B) \, Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


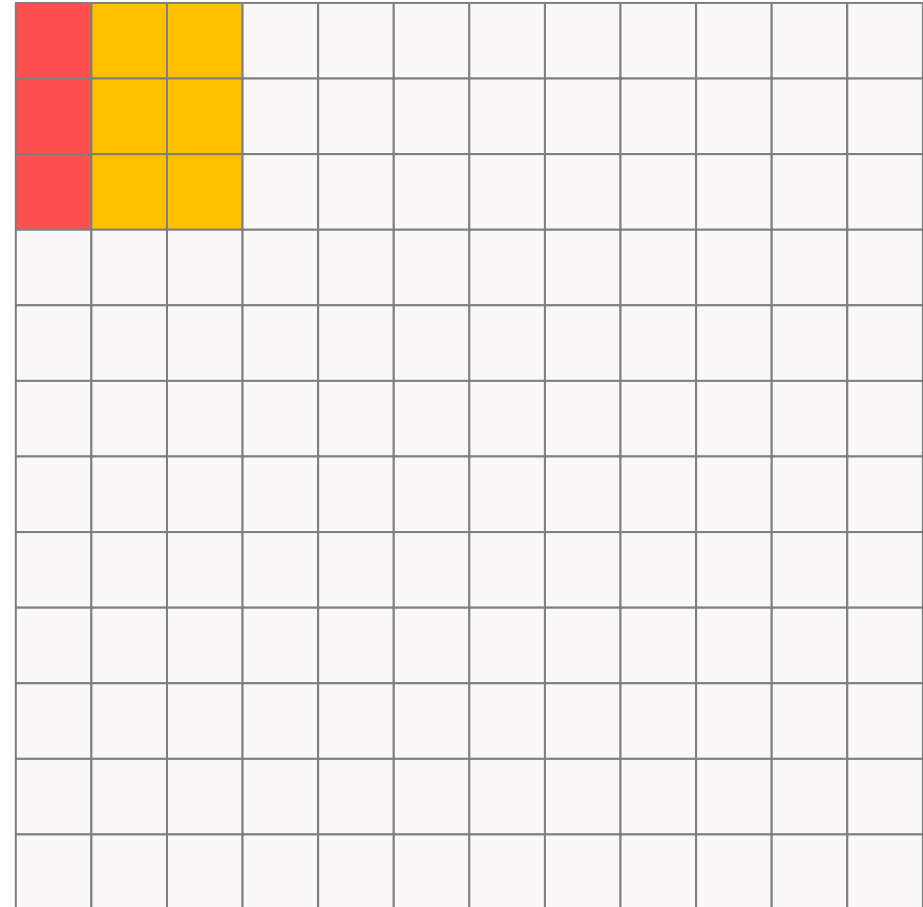
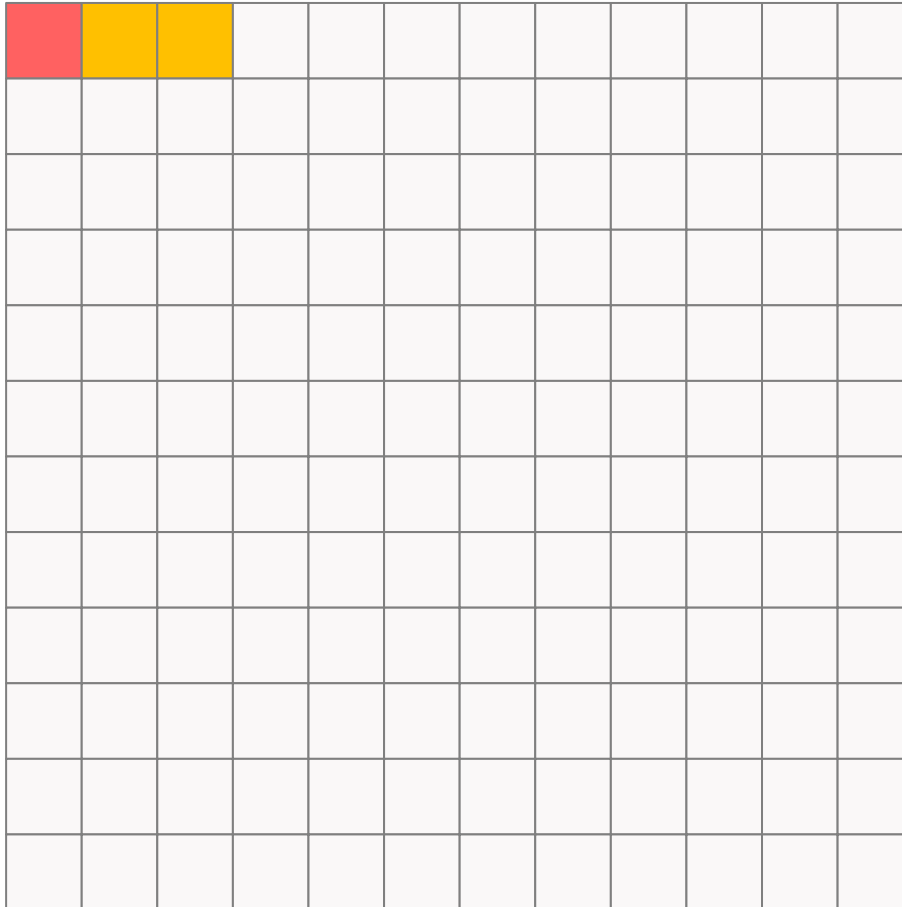
# MULT: Memory Accesses

$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


# MULT: Memory Accesses

$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


# MULT: Memory Accesses

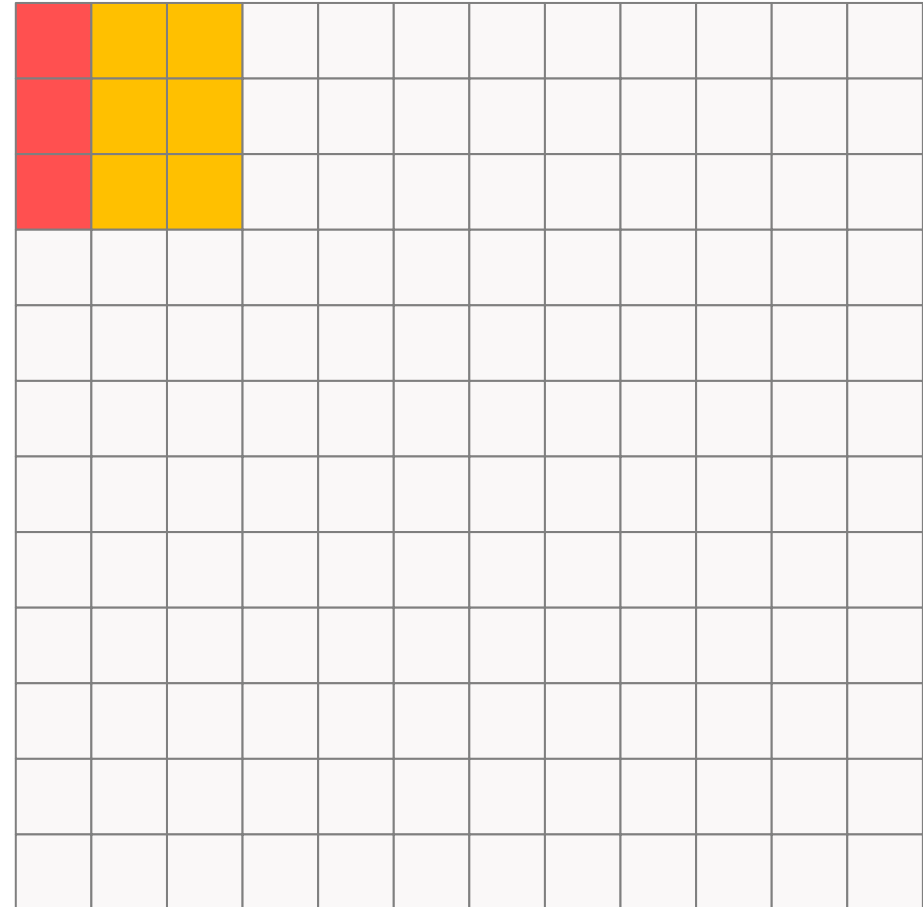
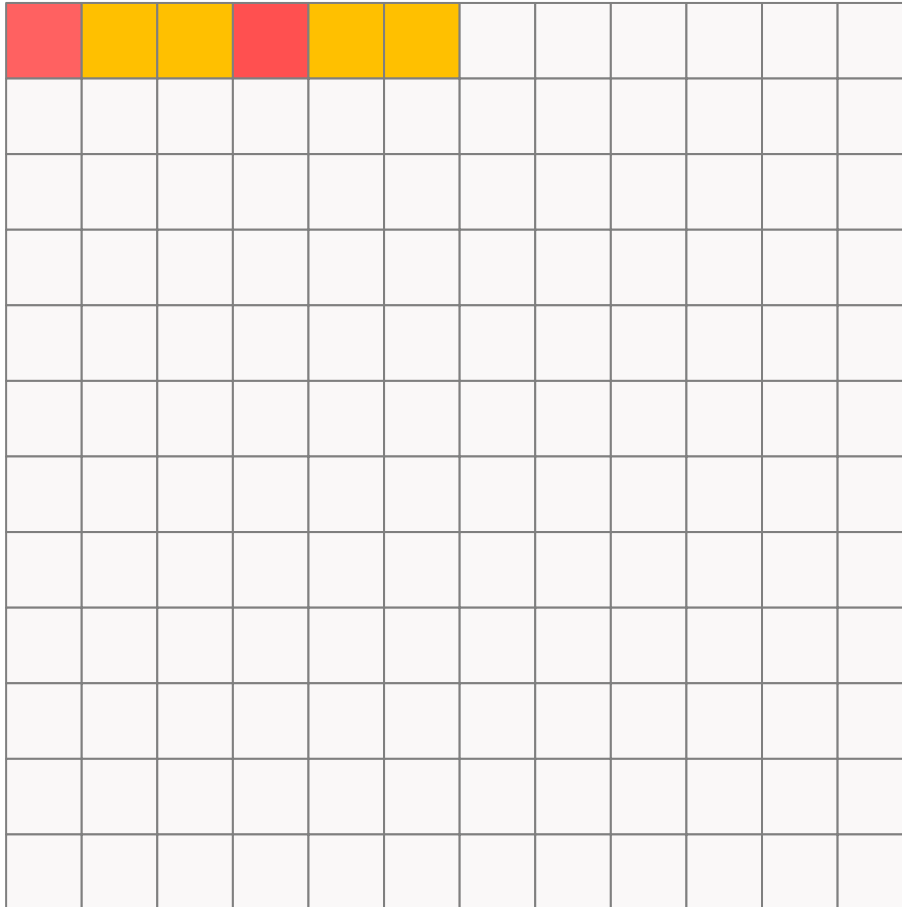
$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \ Y(B+1, j) \ X(i, B+2) \ Y(B+2, j) \ \dots \ X(i, 2B) \ Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

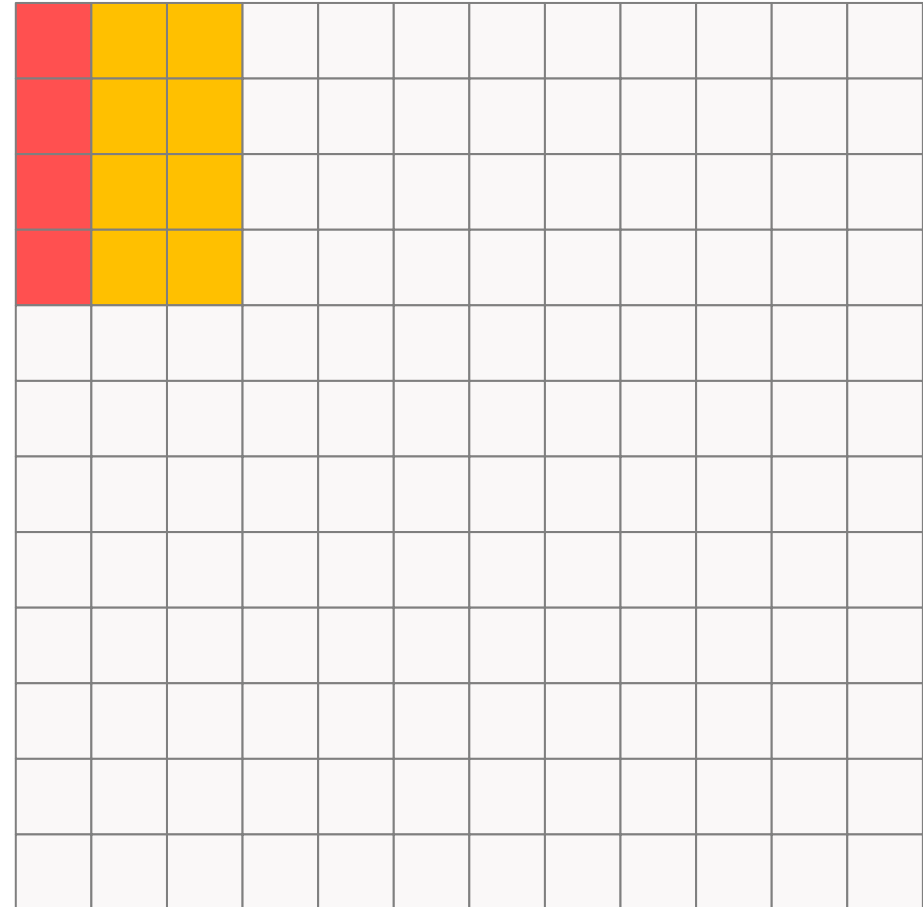
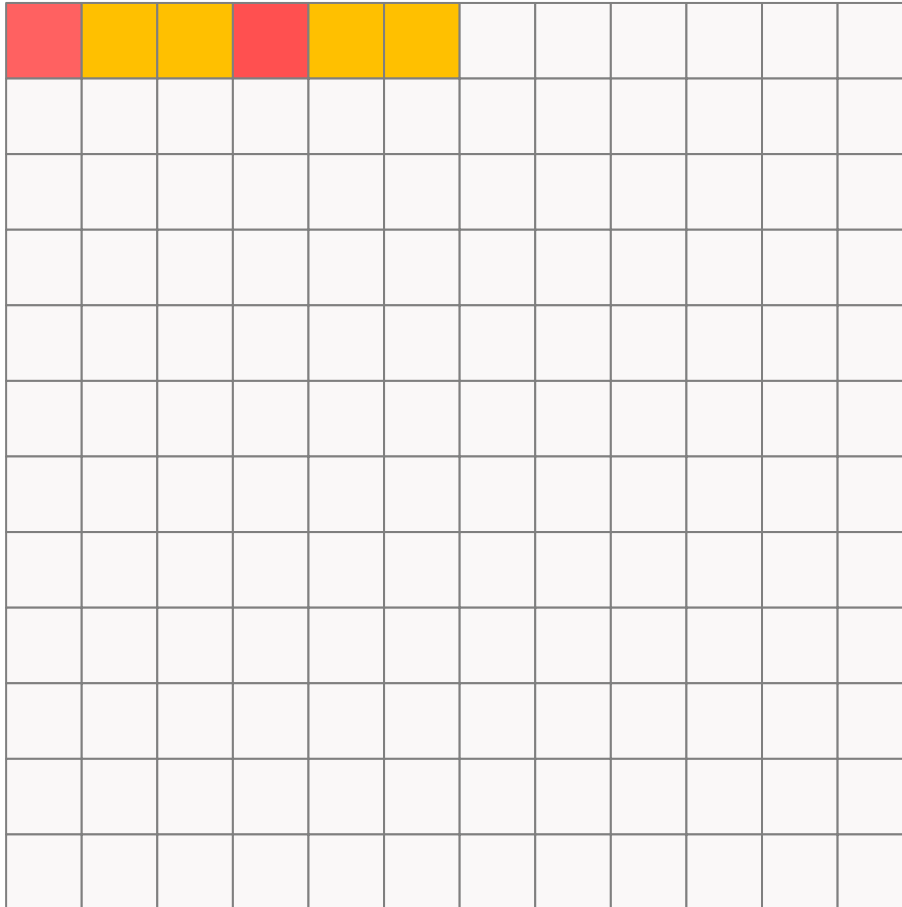


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...



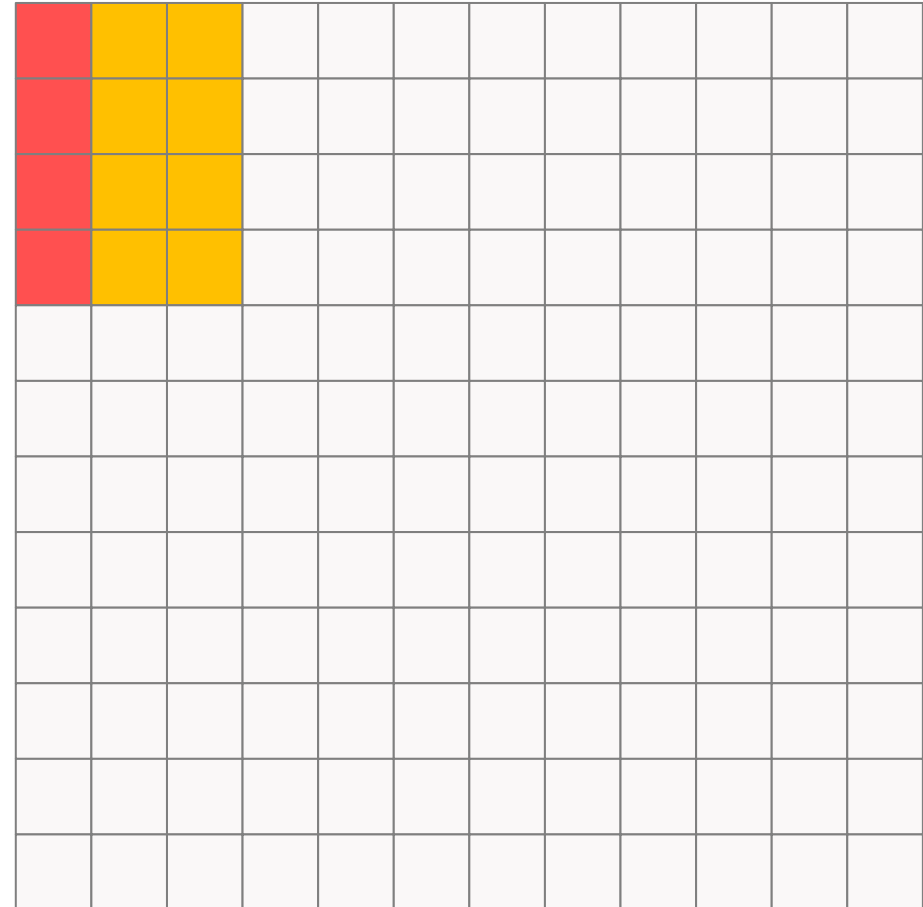
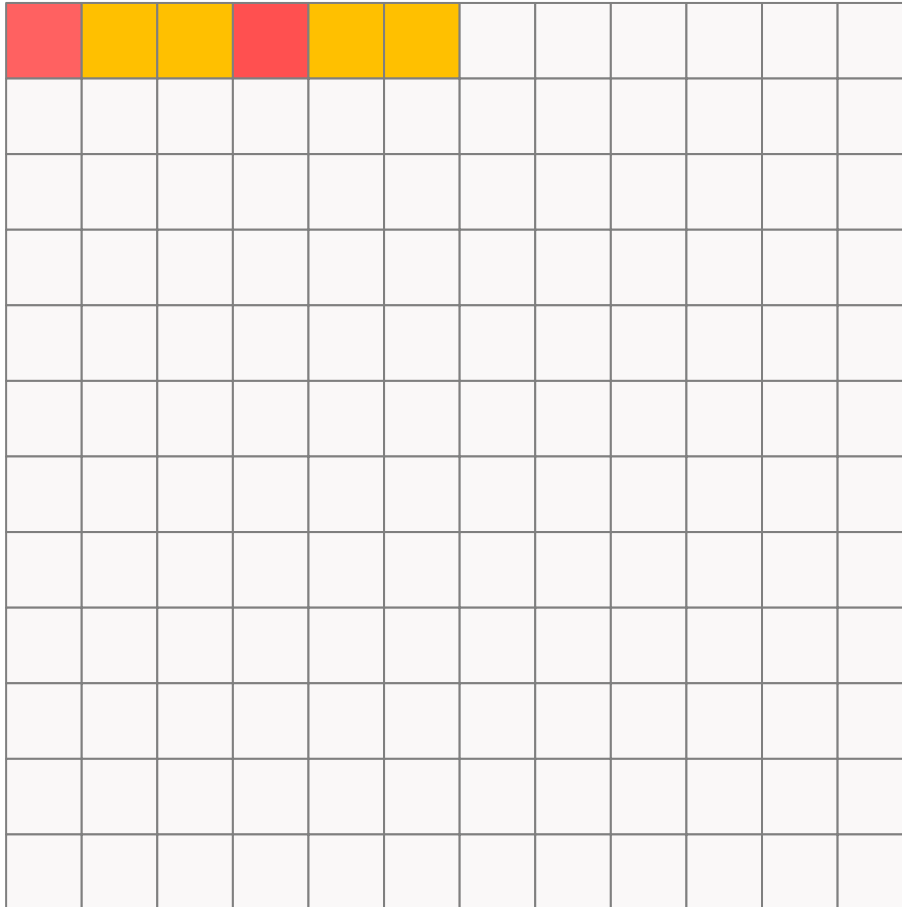


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

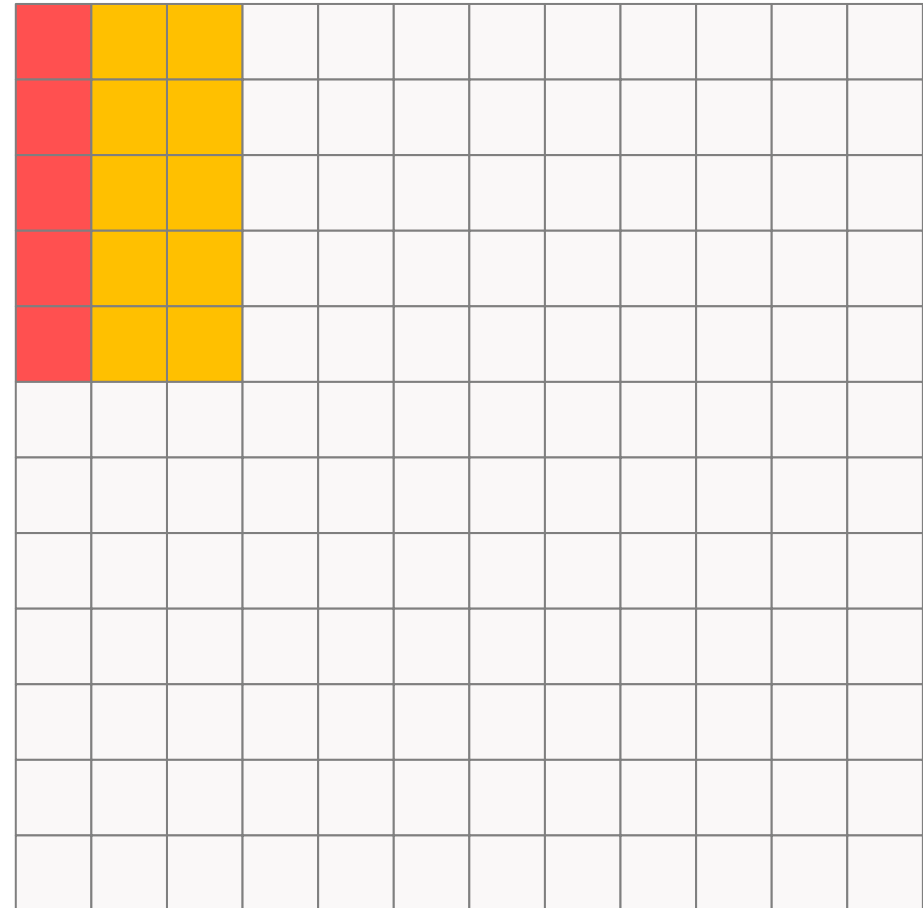
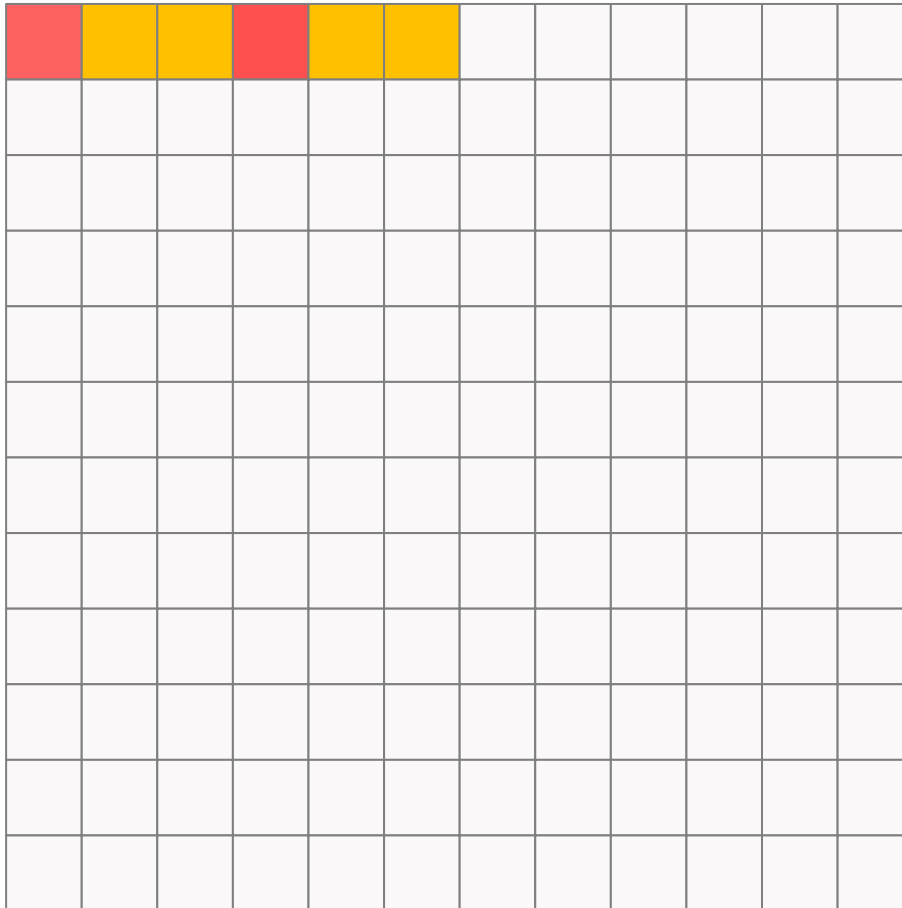


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

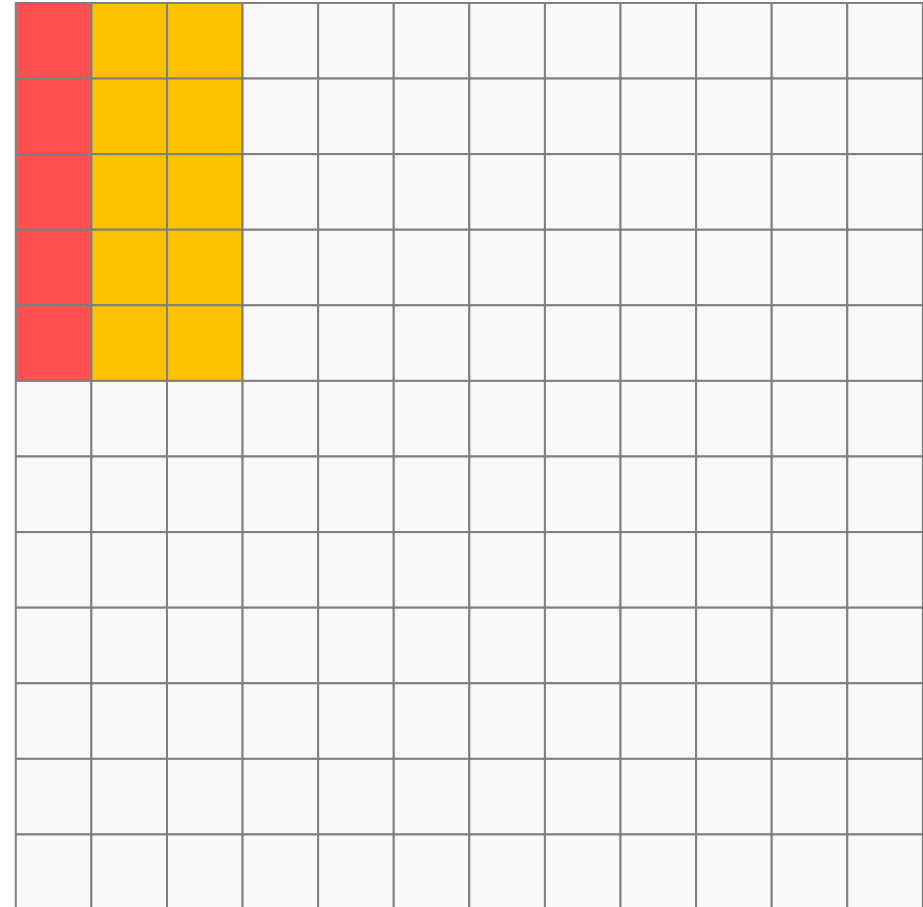
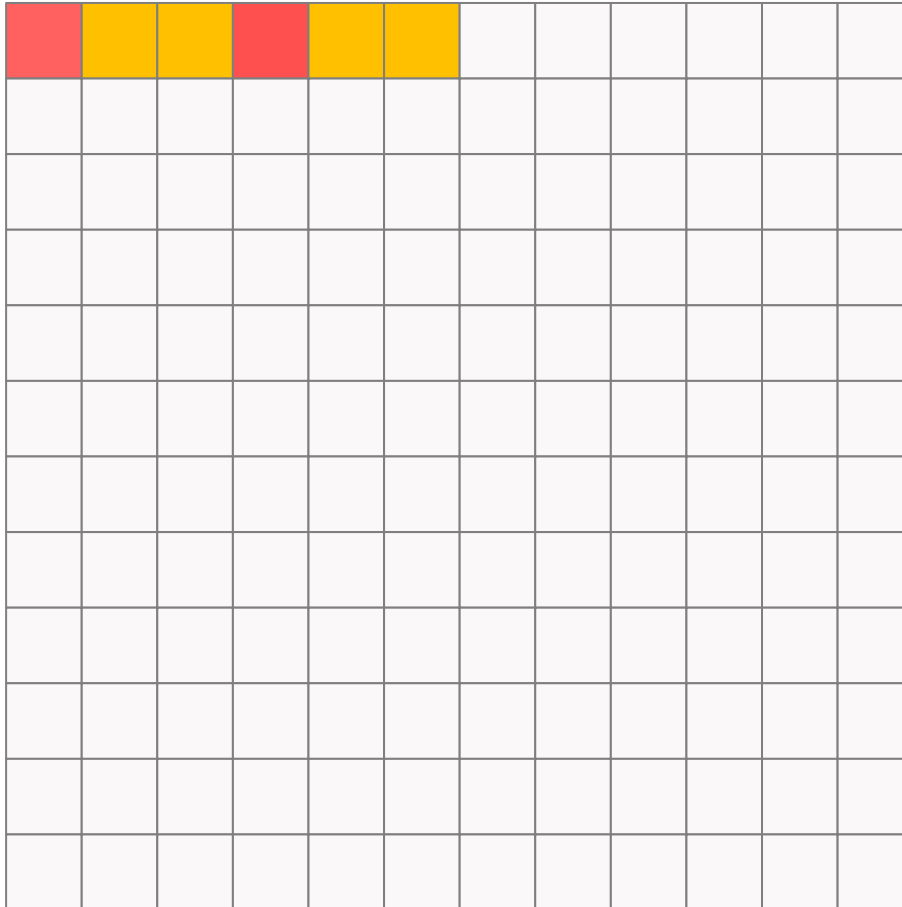


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

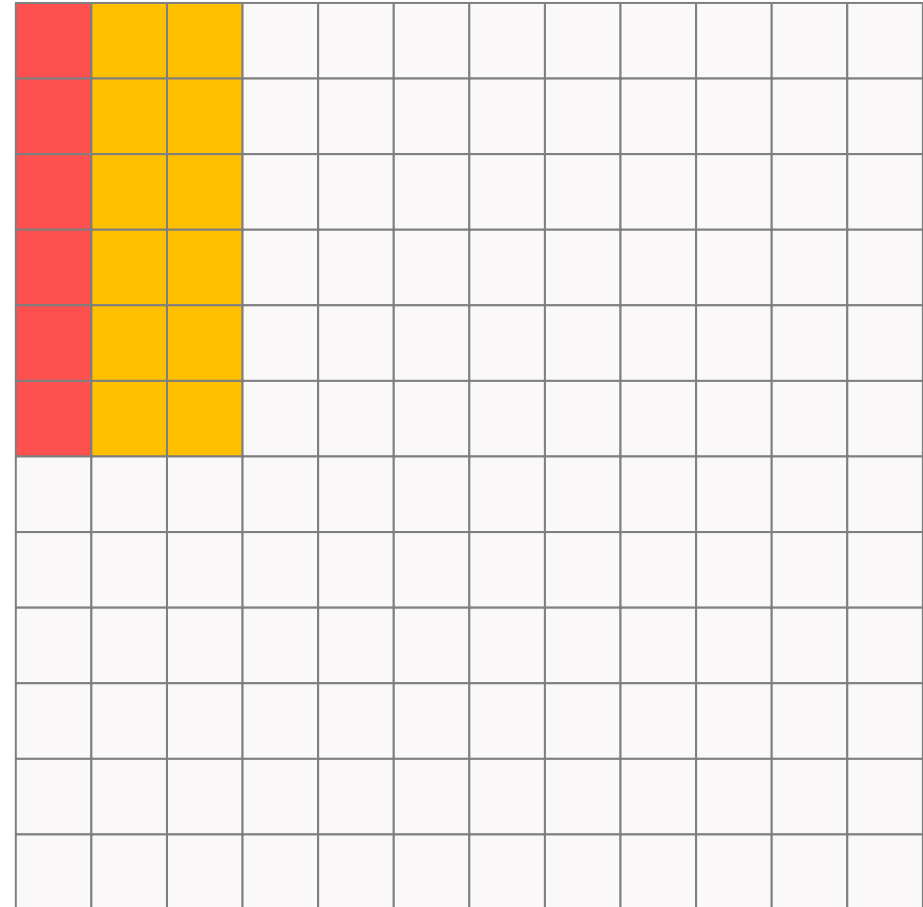
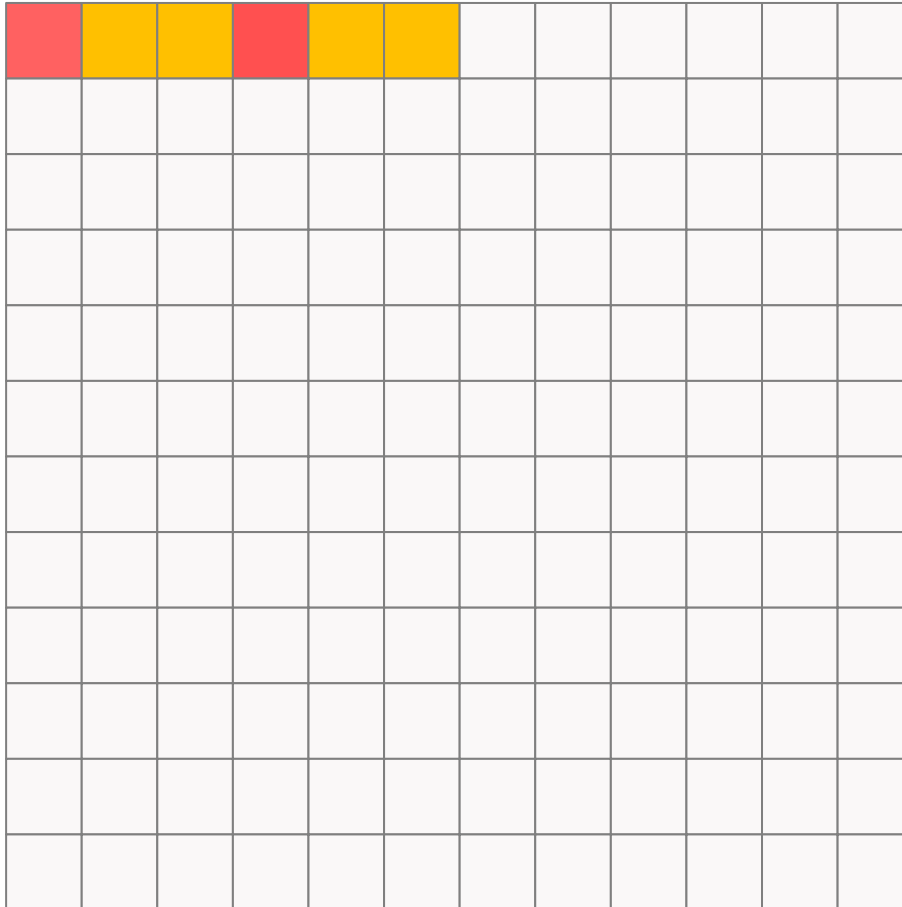


# MULT: Memory Accesses

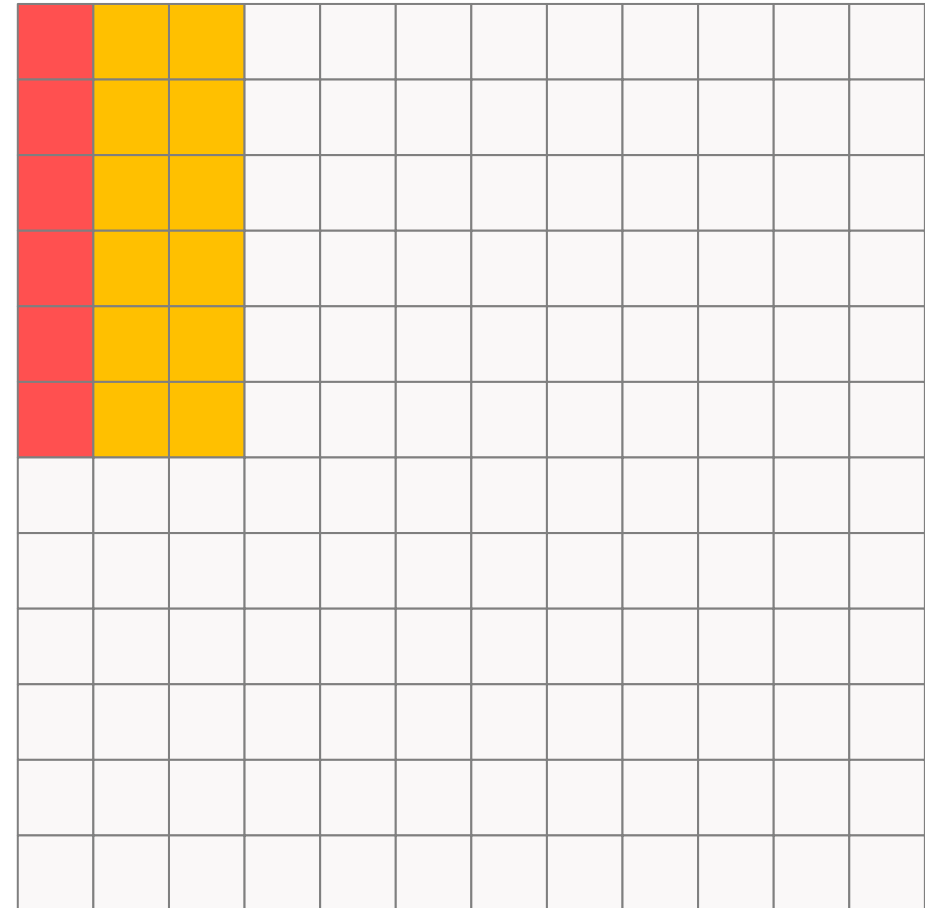
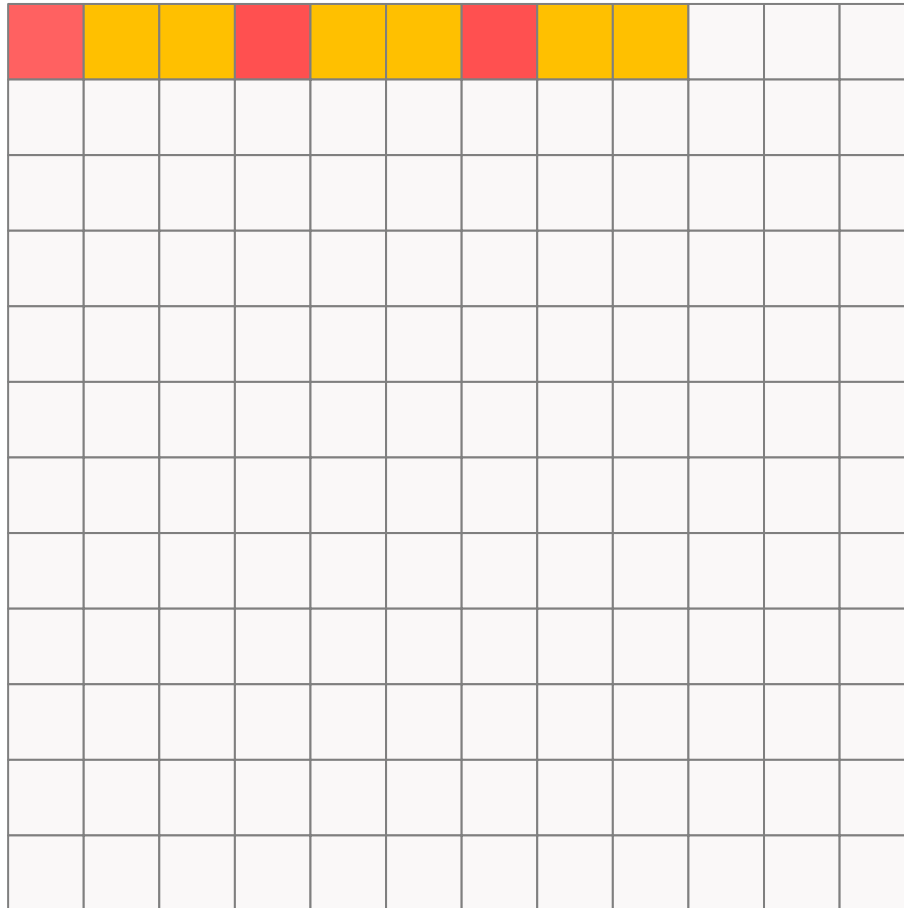
$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...



# MULT: Memory Accesses

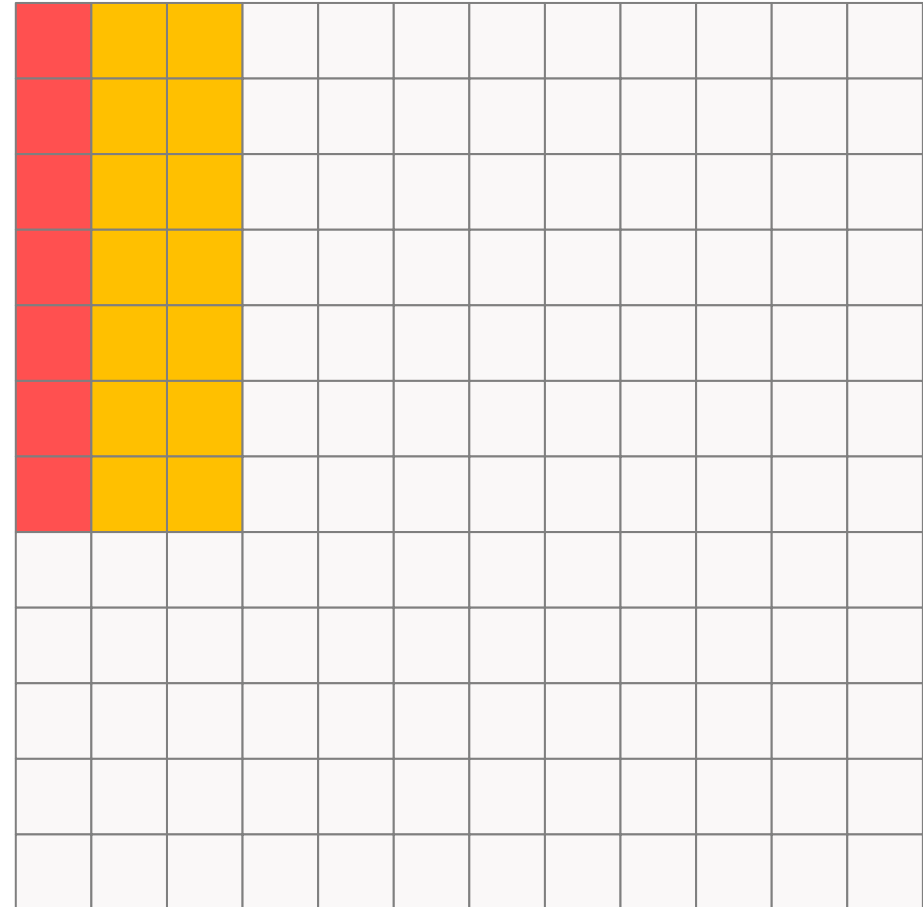
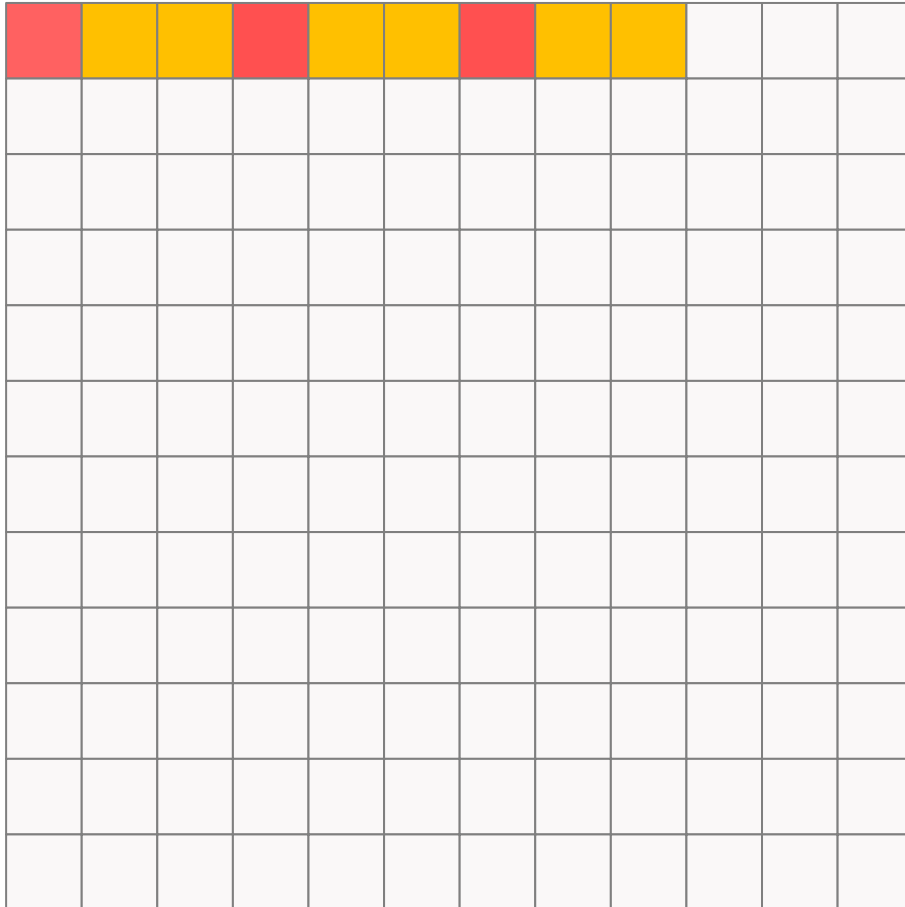
$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \quad Y(B+1, j) \quad X(i, B+2) \quad Y(B+2, j) \quad \dots \quad X(i, 2B) \quad Y(2B, j)$$
$$X(i, 2B + 1) \ Y(2B + 1, j) \ X(i, 2B + 2) \ Y(2B + 2, j) \ \dots \ X(i, 3B) \ Y(3B, j) \ \dots$$


# MULT: Memory Accesses

$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

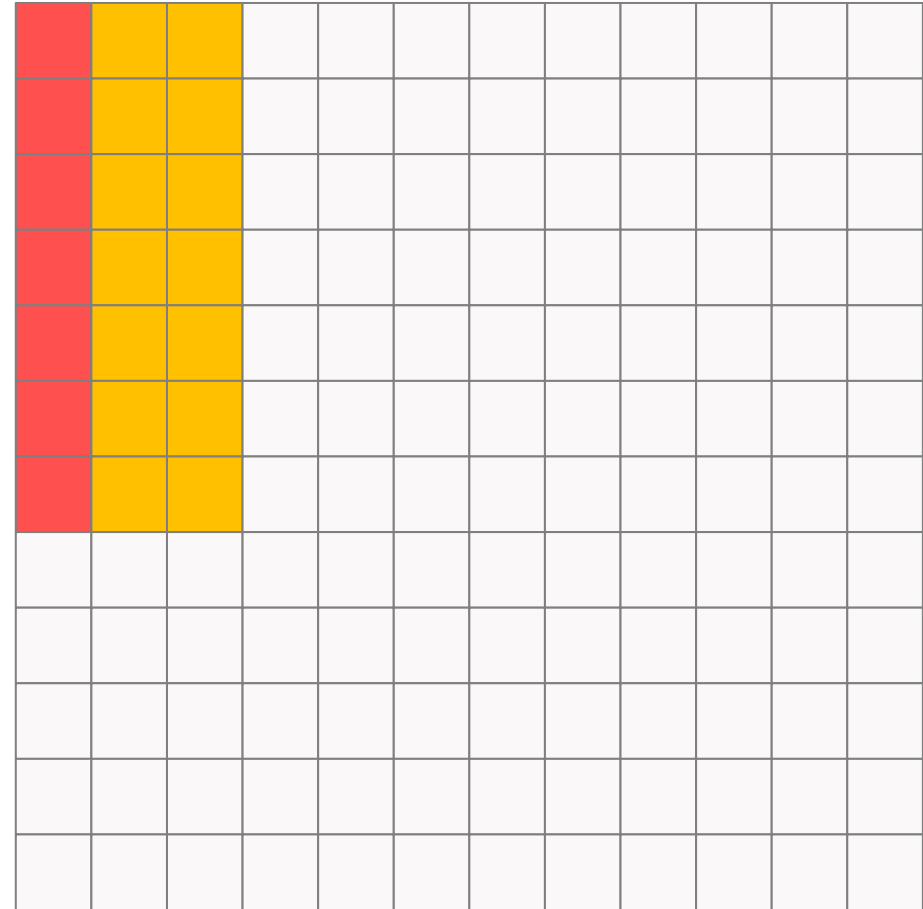
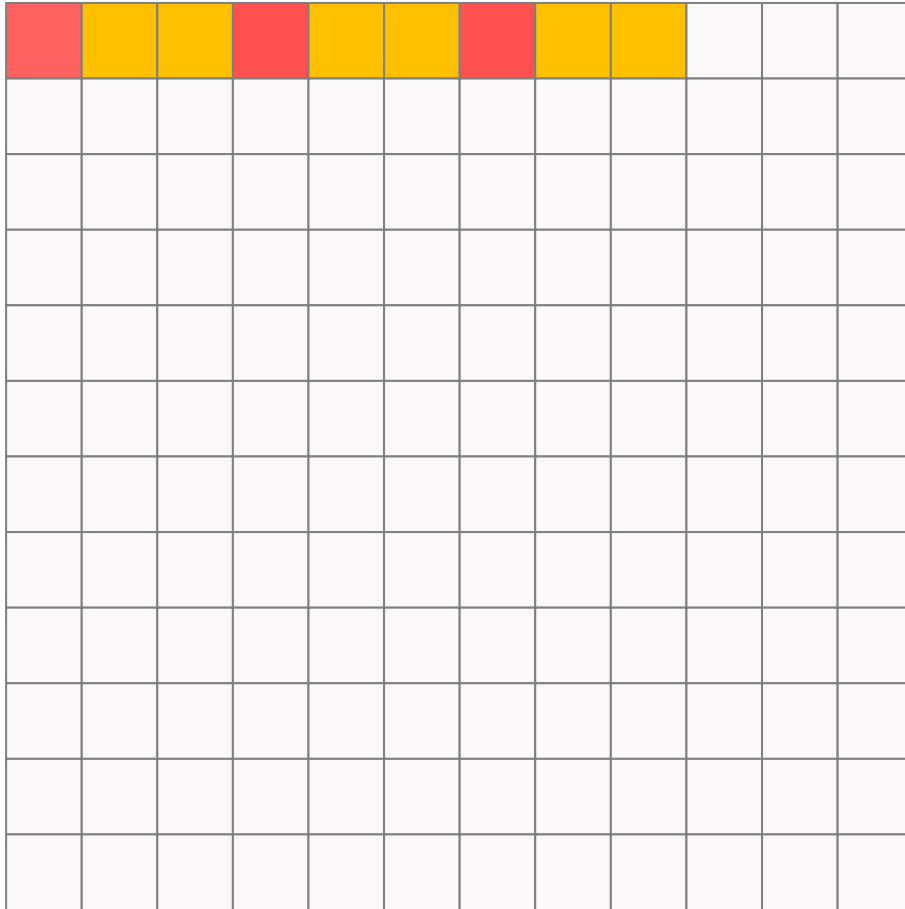


# MULT: Memory Accesses

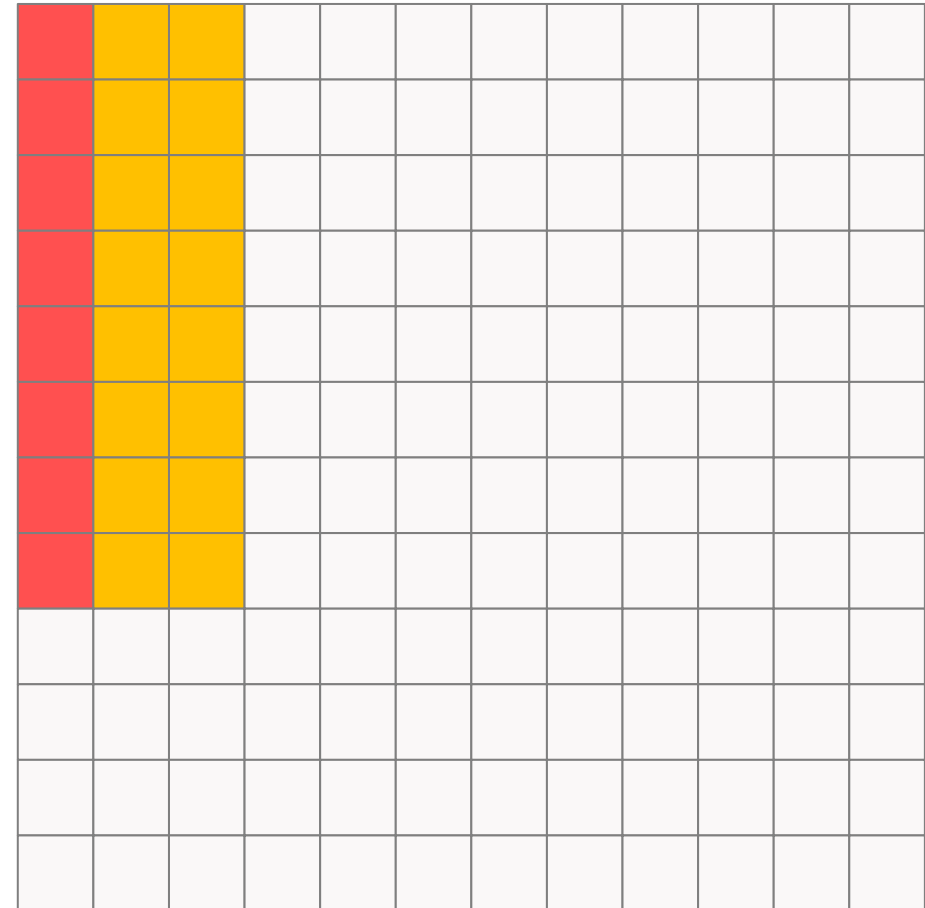
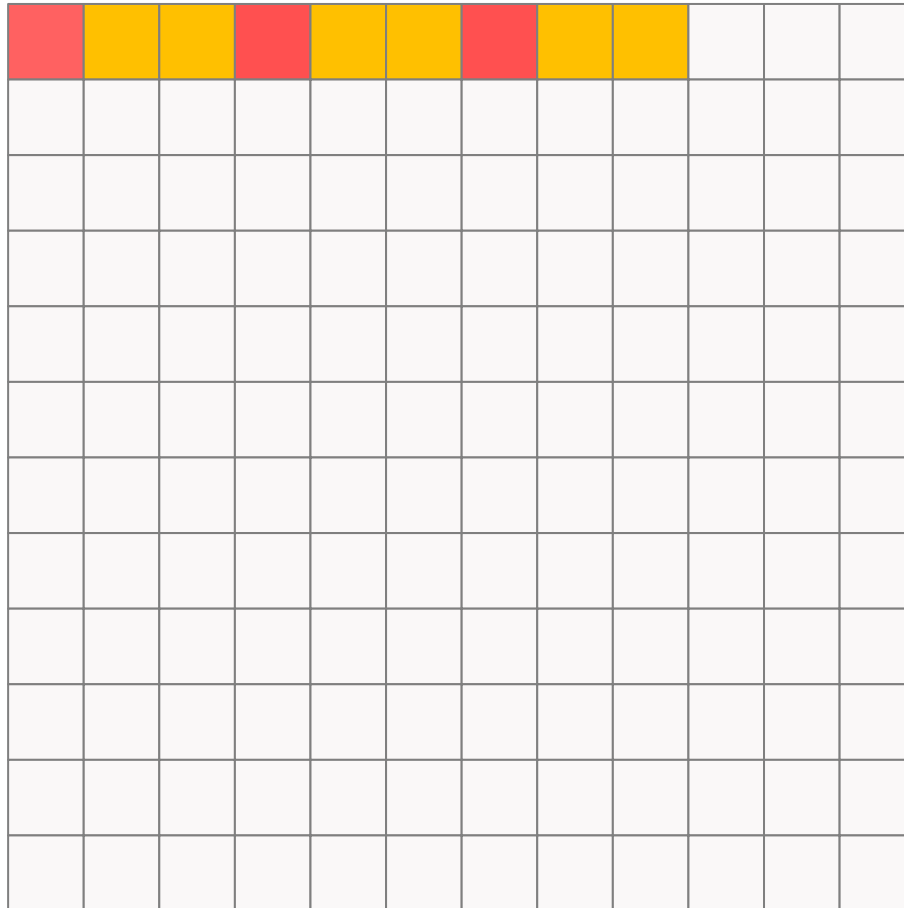
$X(i, 1)$   $Y(1, j)$   $X(i, 2)$   $Y(2, j)$  ...  $X(i, B)$   $Y(B, j)$

$X(i, B + 1)$   $Y(B + 1, j)$   $X(i, B + 2)$   $Y(B + 2, j)$  ...  $X(i, 2B)$   $Y(2B, j)$

$X(i, 2B + 1)$   $Y(2B + 1, j)$   $X(i, 2B + 2)$   $Y(2B + 2, j)$  ...  $X(i, 3B)$   $Y(3B, j)$  ...

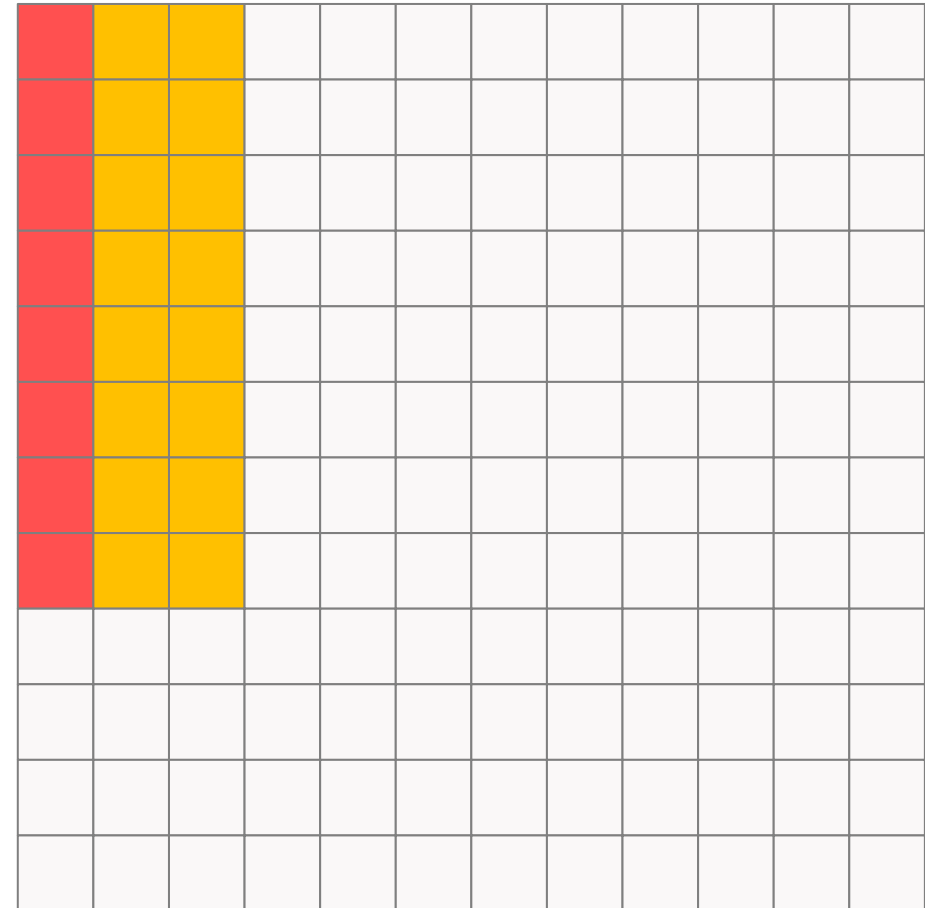
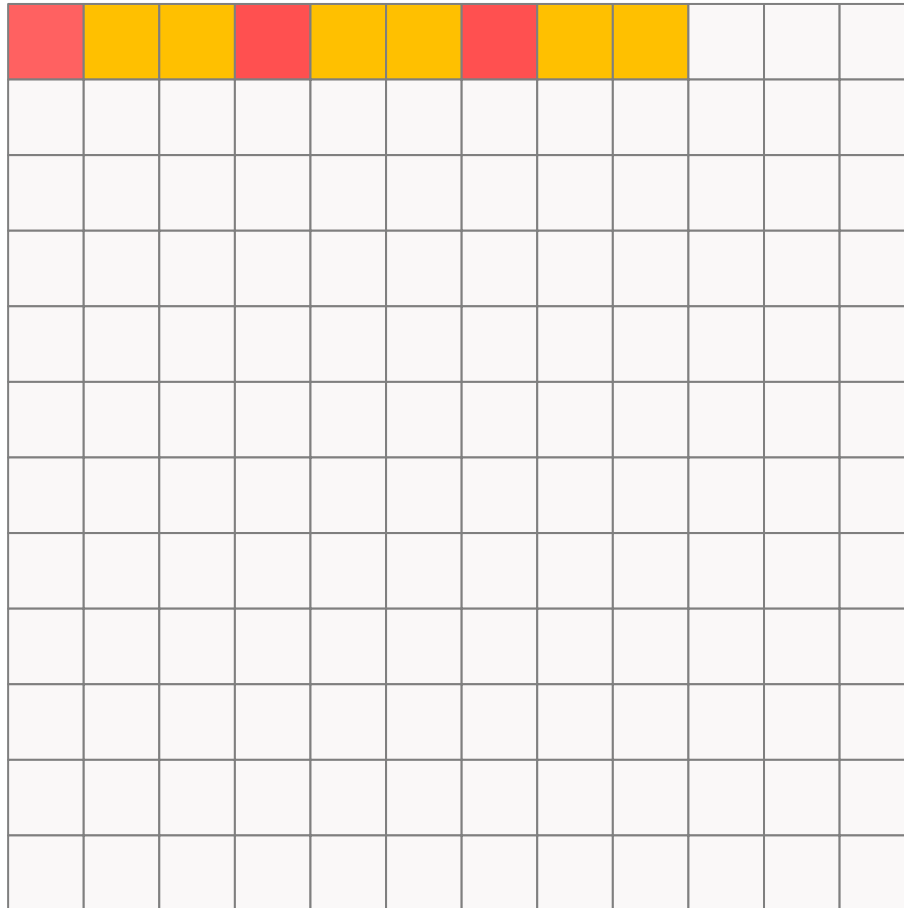


## MULT: Memory Accesses

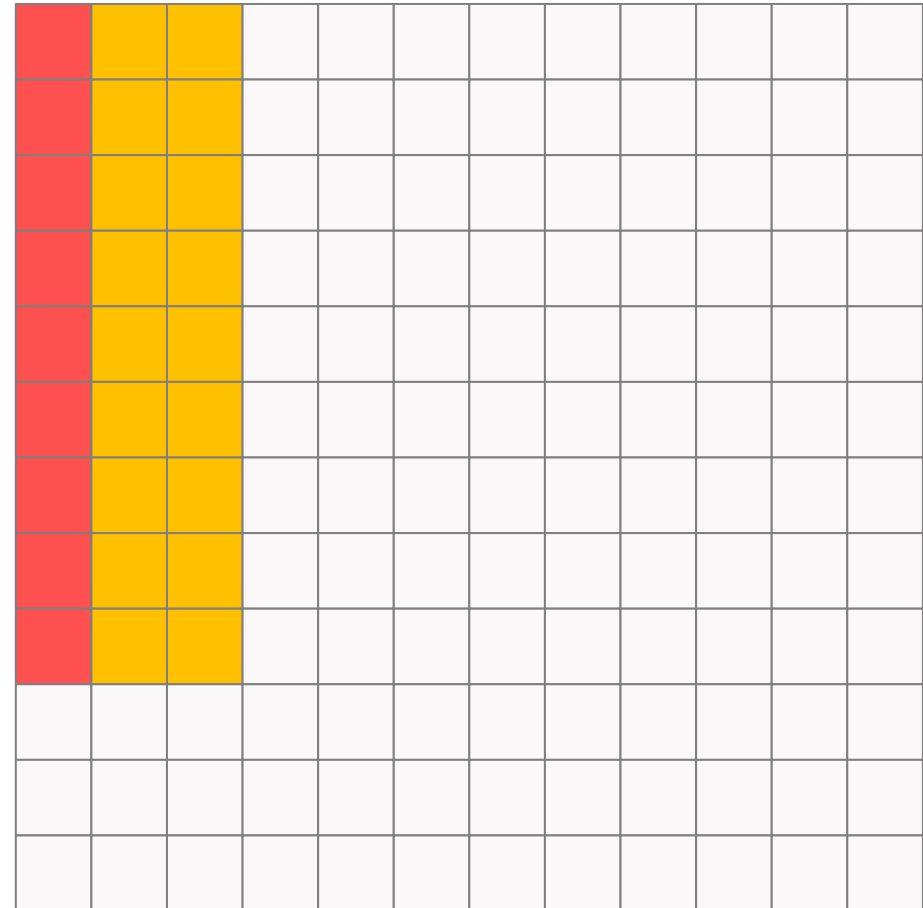
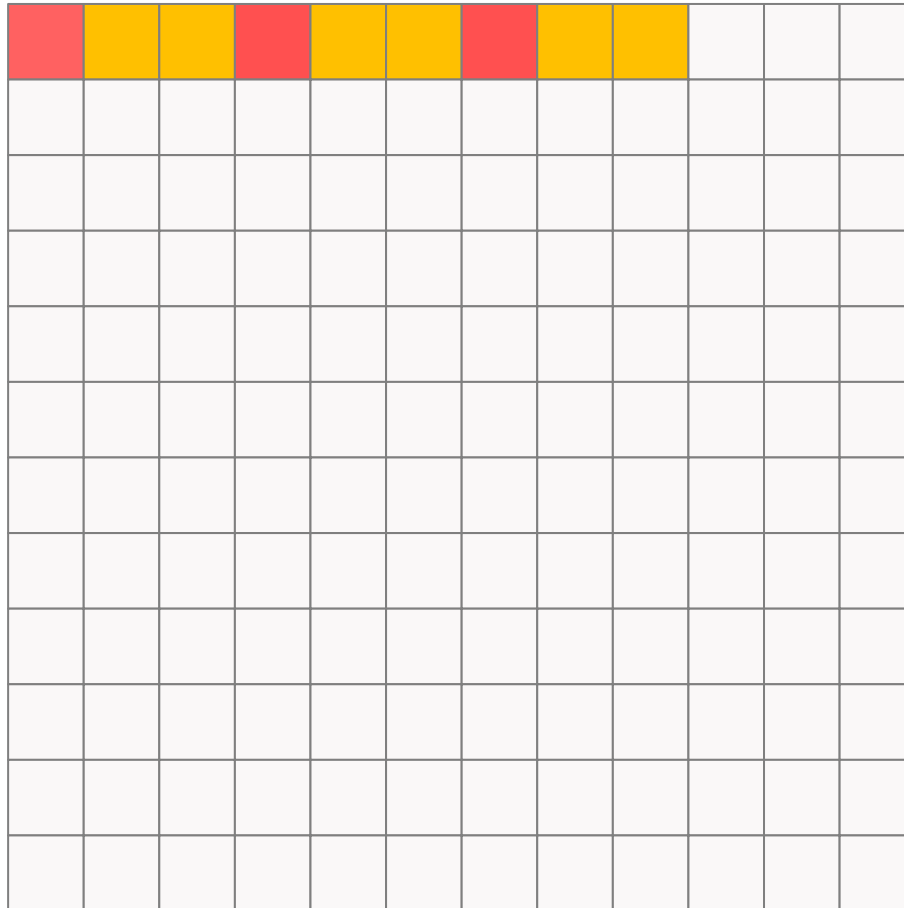
$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \quad Y(B+1, j) \quad X(i, B+2) \quad Y(B+2, j) \quad \dots \quad X(i, 2B) \quad Y(2B, j)$$
$$X(i, 2B + 1) \quad Y(2B + 1, j) \quad X(i, 2B + 2) \quad Y(2B + 2, j) \quad \dots \quad X(i, 3B) \quad Y(3B, j) \quad \dots$$




# MULT: Memory Accesses

$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \quad Y(B+1, j) \quad X(i, B+2) \quad Y(B+2, j) \quad \dots \quad X(i, 2B) \quad Y(2B, j)$$
$$X(i, 2B + 1) \quad Y(2B + 1, j) \quad X(i, 2B + 2) \quad Y(2B + 2, j) \quad \dots \quad X(i, 3B) \quad Y(3B, j) \quad \dots$$


# MULT: Memory Accesses

$$X(i, 1) \quad Y(1, j) \quad X(i, 2) \quad Y(2, j) \quad \dots \quad X(i, B) \quad Y(B, j)$$
$$X(i, B+1) \quad Y(B+1, j) \quad X(i, B+2) \quad Y(B+2, j) \quad \dots \quad X(i, 2B) \quad Y(2B, j)$$
$$X(i, 2B + 1) \mid Y(2B + 1, j) \mid X(i, 2B + 2) \mid Y(2B + 2, j) \mid \dots \mid X(i, 3B) \mid Y(3B, j) \mid \dots$$


# Analysis of MULT

For a gemm operation on three  $n \times n$  matrices, algorithm MULT requires

$$F(n) = 2n^3$$

operations and incurs

$$Q(n; M, B) \approx n^3 \left( 1 + \frac{1}{B} \right) + 2n^2$$

cache misses in the ideal-cache model.

# Analysis of MULT

i,j-Loop      Read Y      Read X      Read/Write Z

$$Q(n; M, B) = n^2 \left( n + \left\lceil \frac{n}{B} \right\rceil + 2 \right)$$
$$\approx n^3 \left( 1 + \frac{1}{B} \right) + 2n^2$$

Assumption: Row-Major Ordering and  $n > M/B$

# MULT-BLOCKED: Blocked Version

---

**Algorithm 4.2:** Given three  $n \times n$  matrices  $Z$ ,  $X$  and  $Y$ , compute the update  $Z \leftarrow Z + X \cdot Y$  using a block-oriented algorithm that employs an ordinary algorithm on  $s \times s$  blocks.

---

```
1 Function  $Z = \text{MULT-BLOCKED}(X, Y, Z, n, s)$ 
2   Partition  $Z = \begin{bmatrix} Z_{11} & Z_{12} & \dots & Z_{1 \frac{n}{s}} \\ Z_{21} & Z_{22} & \dots & Z_{2 \frac{n}{s}} \\ \vdots & \vdots & \ddots & \vdots \\ Z_{\frac{n}{s} 1} & Z_{\frac{n}{s} 2} & \dots & Z_{\frac{n}{s} \frac{n}{s}} \end{bmatrix}$  into blocks  $Z_{ij}$  of size  $s \times s$  and partition  $X$ 
   and  $Y$  in the same way
3   for  $i = 1$  to  $n/s$  do
4     for  $j = 1$  to  $n/s$  do
5       for  $k = 1$  to  $n/s$  do
6          $Z_{ij} \leftarrow \text{MULT}(X_{ik}, Y_{kj}, Z_{ij}, s)$ 
```

---

# Analysis of MULT-BLOCKED

For a gemm operation on three  $n \times n$  matrices,  
algorithm MULT-BLOCKED requires

$$F(n) = 2n^3$$

operations and incurs

$$Q(n; M, B) \approx \frac{n^3}{B\sqrt{M}}$$

cache misses in the ideal-cache model.

# Analysis of MULT-BLOCKED

i,j,k-Loop      Write submatrices to cache, fill all  $M/B$  cache lines

$$\begin{aligned} Q(n; M, B) &\approx \left( \frac{n^3}{s^3} \right) \left( \frac{M}{B} \right) = \left( \frac{n^3}{B} \right) \left( \frac{M}{s * s^2} \right) \\ &= \left( \frac{n^3}{B} \right) \left( \frac{M}{M\sqrt{M}} \right) = \left( \frac{n^3}{B\sqrt{M}} \right) \end{aligned}$$

Assumption:  $M \sim s^2 \Leftrightarrow s \sim \sqrt{M}$ , i.e.  $s$  sufficiently small s. t. matrices fit in cache